

CHAPTER 5. EXPERIMENTAL EVALUATION

5.1 Evaluation Objectives

The evaluation was designed to answer four questions. First, does the lexical scoring and Rocchio feedback pipeline behave deterministically and change holdout rankings in the intended direction under a controlled causal scenario? Second, do the backend modules and database migrations satisfy their automated unit, integration, security, and contract tests? Third, can the main Guest, Candidate, Recruiter, and Administrator workflows execute through the integrated browser application? Fourth, what can be observed about local API latency, authorization boundaries, health, and monitoring without overstating the evidence as production validation?

The evaluation separates algorithmic effectiveness, software correctness, browser workflow acceptance, security checks, and operational health. A pass in one category does not imply a pass in another. In particular, the controlled Rocchio benchmark is not a measurement of hiring quality on organic production data, and a successful backend test suite does not prove that browser, monitoring, or deployment behavior is complete.

Backend, algorithm, frontend build, and Chromium test results were refreshed on July 18, 2026 (ICT). The observed Git HEAD was 65318fb0e0978574c9a04d9e54aecca5ba1eb241, but the worktree contained additional modifications and untracked files. Therefore, the results describe the evaluated working tree and are not reproducible from the commit hash alone unless the complete working-tree state is preserved.

5.2 Evaluation Environment

Table 5.1. Evaluation environment

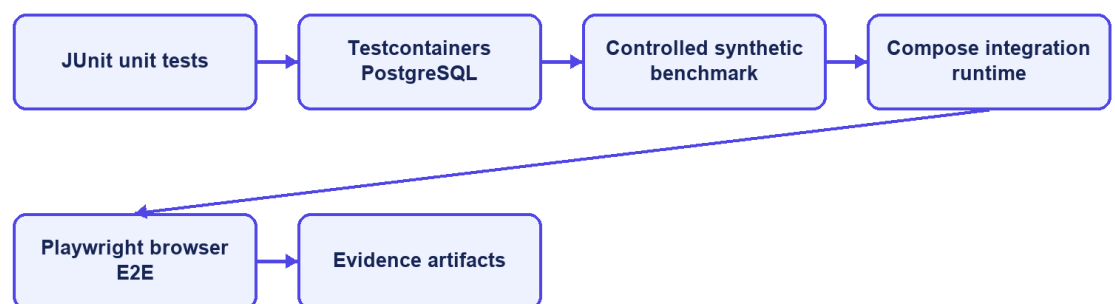
Component	Observed version or configuration
Host operating environment	Windows, PowerShell
Java	21.0.1
Backend framework	Spring Boot 3.2.5
Maven	Repository Maven Wrapper
Node.js / npm	20.16.0 / 10.8.1
Frontend build	Vite 6.4.3, React 18.3, TypeScript 5.9
Docker / Compose	Docker Desktop server 29.5.3 / Compose v5.1.4

Component	Observed version or configuration
Test database	PostgreSQL 16.14 through Testcontainers or Compose
Testcontainers	1.21.4
Browser E2E	Playwright 1.61, Chromium project
Local backend	Host process on port 8080 connected to Compose PostgreSQL at localhost:5433
Local frontend	Vite development server on 127.0.0.1:5173

Docker Desktop was initially unavailable and was started before database-dependent verification. Testcontainers created isolated PostgreSQL containers for backend integration tests. Browser E2E used the existing local CareerFit database, which contained imported and seeded data. The public Job API reported 991 Jobs during the runtime check. This database is not the same dataset as the controlled algorithm benchmark.

The original command catalog is recorded in evidence/CHAPTER5_EVIDENCE_20260703.md. For the July 18 refresh, current Surefire XML reports, Playwright output, the production-build output, backend runtime logs, and evaluation/result.json provide the underlying artifacts.

Evaluation environments



CareerFit IT AutoPilot — thesis diagram

Figure 5.1. Evaluation environments and evidence sources

5.3 Dataset and Evaluation Design

5.3.1 Controlled Algorithm Dataset

AlgorithmEvaluatorTest loads evaluation/controlled-dataset.json. The dataset contains 50 Jobs and 100 unique CVs across IT domains. It defines 300 training pairs and 300 holdout pairs. The file hash observed during every repeated run was 6e935639ba6d3290dca8ad91a35d714c5e30c7e69a59af23ddbcbf89fcc5cc2f2.

The benchmark is intentionally synthetic. Each Job is associated with a training-positive CV and a separate holdout-positive CV sharing a latent skill that is not emphasized in the original JD. The baseline ranks candidates using the original lexical representation. A positive feedback event supplies evidence from the training CV; Rocchio updates the Job vector; and the system reranks candidates. Metrics are then computed on holdout relevance rather than using the same CV that generated feedback as the only success case.

This design tests causal behavior: whether a defined relevance signal can move a related holdout item upward. It does not reproduce organic recruiter behavior, naturally imbalanced applicant pools, demographic variation, adversarial CV writing, or changing labor-market terminology. The deliberately planted latent feature also makes a large improvement possible and should not be interpreted as the expected effect size in production.

5.3.2 Local Runtime Data

The integrated runtime used Flyway seed data and previously imported Job records. It supports realistic API volume and complete application workflows, but it is not a labeled relevance dataset. Consequently, it is suitable for functional/E2E and small latency checks, not for calculating ranking precision or fairness. E2E uses demo Candidate, Recruiter, and Administrator accounts and changes local database state.

5.4 Backend Automated Testing

The complete backend suite was executed through the Maven Wrapper using .mvnw.cmd test. It compiled 127 application source files and 22 test source files, started Testcontainers, validated and applied 15 Flyway migrations, and executed 20 Surefire test suites.

Table 5.2. Backend automated test results

Measure	Result
Test suites	20
Tests	72
Failures	0
Errors	0
Skipped	0
Aggregated Surefire test time	244.659 s
End-to-end command wall time	Approximately 266 s

The suites covered application context startup, API contracts, post-commit execution, authentication, auto-apply, Candidate profile and CV ingestion, feedback authorization, Job service, matching batches, document extraction, quality validation, Rocchio, scoring, settings, TF-IDF, production configuration, and security hardening. AlgorithmEvaluatorTest was included in the same complete suite.

All 72 registered JUnit tests passed in the July 18 run, with no failures, errors, or skips. Flyway 9.22.3 still warned that PostgreSQL 16.14 is newer than the highest version it reports as tested, and some negative tests intentionally logged handled exceptions. The final benchmark log contained no StaleObjectStateException or build failure.

5.5 Controlled Algorithm Results

The benchmark used Rocchio parameters $\alpha=1.0$, $\beta=0.75$, and $\gamma=0.15$. Coverage remained 1.0 before and after feedback. Table 5.3 reports the fresh artifact values.

Table 5.3. Baseline and Rocchio results on the synthetic holdout scenario

Metric	Baseline	After Rocchio	Delta
Precision@5	0.012000	0.172000	+0.160000
Recall@5	0.060000	0.860000	+0.800000
nDCG@3	0.030000	0.830000	+0.800000
nDCG@5	0.037737	0.837737	+0.800000
nDCG@10	0.050424	0.850424	+0.800000
MRR	0.058755	0.842665	+0.783910
HitRate@5	0.060000	0.860000	+0.800000
Coverage	1.000000	1.000000	0

The increase across position-sensitive metrics shows that the controlled positive feedback moved relevant holdout CVs toward the top of the ranking. Precision@5

reaches 0.172 after feedback, so the top-five lists are not composed entirely of labeled relevant items. Recall@5 and HitRate@5 reach 0.86 in the designed scenario, while 14 percent of query cases still do not place the expected holdout item within the first five results.

The July 18 full-suite run used dataset hash 6e935639ba6d3290dca8ad91a35d714c5e30c7e69a59af23ddbcbf89fcc5cc2f2. It produced baseline nDCG@5 of 0.037737056145, Rocchio nDCG@5 of 0.837737056145, and a delta of 0.80. AlgorithmEvaluatorTest completed in 212.2 seconds inside the full suite. These values describe the current code and dataset; they replace the earlier July 3 metric snapshot.

Controlled benchmark: baseline versus Rocchio

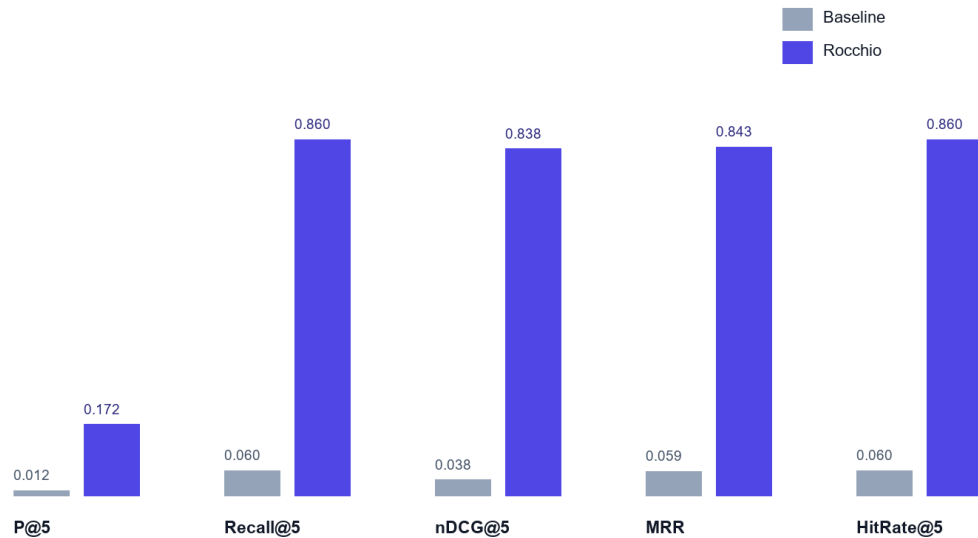


Figure 5.2. Baseline and Rocchio benchmark metrics at $K = 5$

5.6 Concurrency Observation in Benchmark Runs

The final isolated benchmark completed without `StaleObjectStateException`. Feedback learning is registered after transaction commit, and benchmark setup removes prior Matchings in bulk and clears the persistence context before recomputation. This makes the repeated baseline-versus-learned measurement deterministic within the controlled test environment.

Earlier benchmark repetitions exposed a background persistence conflict even when foreground assertions passed. The current run no longer logged that exception because feedback learning and matching begin after the initiating transaction commits

and the benchmark clears prior Matching state before recomputation. This distinction remains important: test assertions and background operational cleanliness must both be checked.

The implemented correction controls asynchronous transaction timing and checks the final logs for background failures. Production work should still define retry, conflict, timeout, and recovery policies, and CI should continue to fail when uncaught background exceptions appear even if foreground JUnit assertions pass.

5.7 Frontend Build Evaluation

npm run build performs TypeScript checking with no output emission and then builds the production assets with Vite 6.4.3. The July 18 command completed successfully and transformed 2,417 modules.

Table 5.4. Frontend build artifacts

Artifact	Uncompressed size	Gzip size
index.html	1.37 kB	0.67 kB
Main CSS	65.09 kB	12.89 kB
Icons chunk	19.25 kB	4.48 kB
Query chunk	39.60 kB	12.18 kB
React chunk	180.98 kB	59.54 kB
Application chunk	217.39 kB	58.05 kB
Charts chunk	378.44 kB	112.04 kB

Manual Rollup chunking separated React, query, chart, and icon dependencies. The largest generated JavaScript chunk was 378.44 kB, below Vite's 500 kB warning threshold. No browser performance profile was collected, so bundle size is reported as build evidence rather than a direct page-load measurement.

5.8 Browser End-to-End Evaluation

The Playwright Chromium project ran against the Vite frontend, host backend, and Compose PostgreSQL. All 20 tests across the P0 workflow, backend-contract, and resilience suites passed in 34.1 seconds.

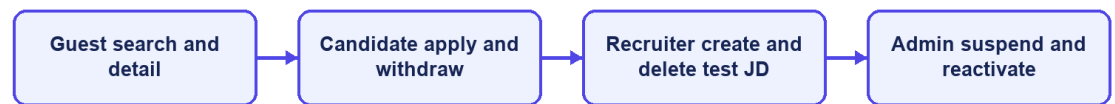
Table 5.5. Chromium P0 workflow results

Scenario	Main verification	Result
Guest search and Job detail	Public search API, Job cards, detail navigation, login-to-apply control	Passed

Scenario	Main verification	Result
Candidate apply and withdraw	Login, personalized Jobs, application creation, history, withdrawal	Passed
Recruiter creates a JD	Recruiter login, creation form, API response, new Job visible	Passed
Administrator suspends/reactivates a user	Admin login, user operation, state transition and restoration	Passed
Candidate and authentication contracts	Passwordless request, settings persistence, recommendations, CV polling, magic-link completion, and session restore	Passed
Resilience and role navigation	Retryable API errors, feedback contract, employer/similar-job data, desktop header, and role-route rendering	Passed

The result covers the four original role workflows together with passwordless request, settings persistence, recommendations, role-route rendering, CV polling, magic-link completion, session restoration, retryable errors, feedback contracts, employer details, similar Jobs, and desktop navigation. It is not a full UAT study: no independent participants performed tasks, seeded credentials were used, and Firefox and WebKit were not run. The Recruiter flow now removes the Job it creates, although the local database still contains earlier test and imported records.

P0 end-to-end coverage



CareerFit IT AutoPilot — thesis diagram

Figure 5.3. P0 end-to-end workflow coverage

5.9 Security-Oriented API Checks

Small negative checks were executed against the integrated backend to verify public access, missing authentication, invalid authorization scheme, valid Candidate authentication, and role denial.

Table 5.6. API authorization observations

Request	Expected	Observed	Result
Public Job search without token	200	200	Passed
Candidate CV endpoint without token	401	401	Passed
Current-account endpoint with invalid Basic scheme	401	401	Passed
Current-account endpoint with valid Candidate bearer token	200	200	Passed
Administrator dashboard with Candidate bearer token	403	403	Passed

These checks confirm selected URL-layer outcomes, not a complete penetration test. They do not evaluate token theft, cross-site scripting, CV malware, rate limiting, secret rotation, SQL-injection tooling, or every ownership boundary. The email-action flow now uses hashed storage and confirm-then-POST redemption, but the broader production-security topics remain outside the verified evidence.

5.10 Runtime Health, Monitoring, and Local Latency

The public Job API returned HTTP 200. The aggregate /actuator/health endpoint and the liveness and readiness groups returned HTTP 200 with status UP. Mail health is disabled when application mail is disabled, preventing an intentionally absent local mail provider from making aggregate health misleading. Prometheus metrics remained available in the local evaluation profile.

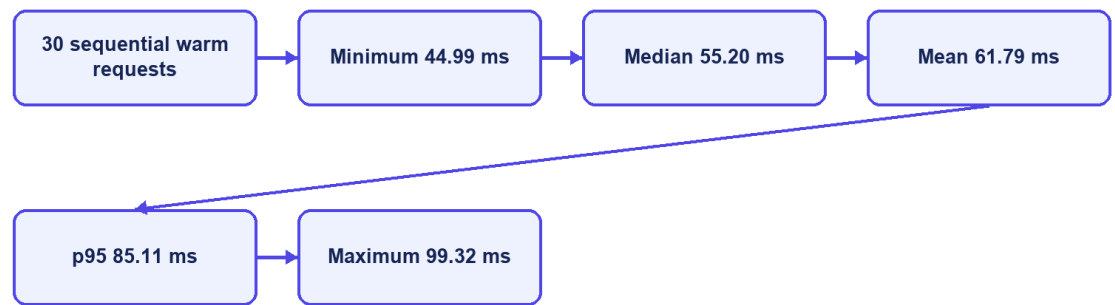
Table 5.7. Runtime observations

Endpoint or check	Observation
/api/jobs/search?page=0&size=20	200
/actuator/health	200, UP
/actuator/health/liveness	200, UP
/actuator/health/readiness	200, UP
/actuator/prometheus	200

The final runtime check returned HTTP 200 and status UP for aggregate health, liveness, and readiness. Mail health is disabled when application mail is disabled, so the absence of a local mail provider no longer makes aggregate health misleading. Production monitoring would still require protected component details, alerting, and an external mail check when email is enabled.

Thirty sequential warm requests to the public Job search endpoint produced a mean of 61.79 ms, p50 of 55.20 ms, p95 of 85.11 ms, minimum of 44.99 ms, and maximum of 99.32 ms. This was a single-client localhost sample without concurrent load, controlled warm-up, network latency, repeated batches, or resource monitoring. It demonstrates only that the observed local endpoint responded consistently in this small sample; it is not a throughput or capacity benchmark.

Observed local Job-search latency



CareerFit IT AutoPilot — thesis diagram

Figure 5.4. Local Job-search latency distribution

5.11 Evaluation Summary by Research Question

Table 5.8. Answers supported by current evidence

Question	Evidence-supported answer
Does Rocchio change holdout ranking in the intended direction?	Yes for the controlled synthetic causal dataset; the same large metric delta was reproduced across three runs.
Is the backend automated suite passing?	Yes: 72/72 registered tests passed, and the final benchmark log contained no optimistic-lock exception.
Do the main browser workflows execute?	All 20 Chromium workflow, contract, and resilience tests passed; Firefox/WebKit and participant UAT remain incomplete.
Are selected authorization boundaries enforced?	The five observed public/authentication/role checks returned expected statuses; this is not comprehensive security validation.
Is the runtime fully healthy?	Yes in the evaluated local profile: aggregate health, liveness, readiness, and the core API returned HTTP 200. This is not production monitoring validation.
Is performance production-ready?	Not evaluated. Only a small sequential localhost latency sample was collected.